Practicumopdracht 1

Introductie

Welkom bij het eerste kwantitatieve practicum van M&T I! In dit practicum ga je leren hoe je R kan gebruiken voor data analyse. Je zult jouw eerste stappen maken in het bekijken, prepareren en analyseren van data. Voordat we aan de slag gaan, volgen eerst een aantal instructies.

Als je nog niet in *Visual* mode werkt, klink dan eerst linksboven in jouw scherm op Visual in plaats van Source. Dit werkt een stuk makkelijker.

Voorbereiding

In het quantcrim handboek staat beschreven welke voorbereiding je nodig hebt om deze practicumopdracht te kunnen maken. Dit zijn de volgende dingen:

- Het installeren van R en Rstudio
- Het installeren van de benodigde packages
- Het bestuderen van de stof van week 4 (Data beschrijven)
- Het voltooien van de Codecademy cursus

Als het goed is, heb je deze dingen al gedaan. Mocht dat niet zo zijn, dan moet je hiermee eerst aan de slag. Anders zul je niet in staat zijn het practicum te maken.

Opdracht maken

Je maakt de opdracht in deze zogenoemde .Rmd file. Dit is een bestand waarin je tekst kan typen en R code apart kan uitvoeren. Dat is handig om zowel analyses uit te voeren en daarover te rapporteren. De opdracht zal aangeven wanneer jij aan de beurt bent om iets doen. Dit gaat hoofdzakelijk om twee dingen: het typen van R code en het geven van een antwoord.

R code typ je in een veld zoals hieronder weergeven. Deze blokken waar je code invult, worden ook wel *code chunks* genoemd.

```
# hier voer je R code in
```

Er staan het volgende wanneer je een antwoord moet geven:

> [Typ hier jouw antwoord]

Verder werken we bij de meeste opgaven volgens het principe van *see one, do one, teach one*. Je krijgt eerst een voorbeeld te zien van hoe het moet en daarna moet je het zelf proberen. Als je daarin geslaagd bent, kun je eventueel een medestudent helpen. Iets doceren is de beste manier om te leren!

Een ander belangrijk principe is dat je zo min mogelijk code kopieert en plakt, maar het juist zelf typt. Typen is een actieve handeling terwijl kopiëren/plakken passief is. Je zult R veel sneller leren als je zelf actief typt en daarbij nadenkt over wat hetgene betekent dat je aan typen bent.

R is tenslotte een programma dat gevoelig is voor hoofd- en kleine letters, leestekens en typfouten. Als je code niet werkt, check dan eerst goed of je nergens een foutje hebt gemaakt.

Op het eind van de opdracht maak je een pdf van dit bestand en lever je dit in via Canvas om aan de **inspanningsverplichting** te voldoen (**deadline: vrijdag 23:59**). Instructies daarvoor staan op het eind.

Data laden

Aan het begin van een analyse moeten we altijd als eerste de relevante packages laden. Die staan hieronder weergeven in het codeblok. Als het goed is, heb je deze al vooraf geïnstalleerd (zie Installatie). De code zal niet werken als je dit niet hebt gedaan.

Je kunt de code selecteren en 'runnen' (Ctrl + Enter) of je klikt op het driehoekje rechtsboven in het codeblok. Dit runnen zul je bij veel codebloks moeten doen. Let dus goed op hoe het moet.

Eventuele waarschuwingen in de output kun je hier negeren.

```
# laden packages
library(tidyverse)
```

```
## Warning: package 'ggplot2' was built under R version 4.4.3
```

```
## Warning: package 'dplyr' was built under R version 4.4.3
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.1     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.2     v tibble    3.2.1
## v lubridate 1.9.3     v tidyr     1.3.1
## v purrr     1.0.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(modeest)
library(plotrix)
```

```
## Warning: package 'plotrix' was built under R version 4.4.3
```

```
library(rmarkdown)
```

```
## Warning: package 'rmarkdown' was built under R version 4.4.2
```

De volgende stap is de dataset te laden die we gebruiken voor onze analyses. Dit is de synthetische dataset gebaseerd op Het Grote Uitgaansonderzoek, zoals beschreven onder Data. Daarvoor moet je eerst de dataset data_uitgaan.rds downloaden via Opdrachten als je dat nog niet gedaan hebt. Sla deze dataset op in je Bureaublad.

Hieronder staat voorbeeldcode van hoe je data laadt in R. Deze code hoef je niet te runnen (zal ook niet werken).

```
# laden data
data_uitgaan <- read_rds("C:/Users/gijs_/Desktop/data_uitgaan.rds")
```

Wat zie je hierboven?

`# laden data` is een beschrijving van wat de regel daaronder doet. We gebruiken hashtags in R om toelichting te geven over wat een stukje code doet. Zoals je hebt gezien in het quantcrim handboek, kan dit zowel boven als naast de code worden gezet.

`data_uitgaan` is de naam die we het aan dataframe geven. Die is in dit geval hetzelfde als de externe dataset.

`read_rds()` is de functie om de externe dataset `data_uitgaan.rds` te laden. Je zou deze functie kunnen zien als "lees een .rds bestandstype in"

`"C:/Users/gijs_/Desktop/data_uitgaan.rds"` is de locatie waar de externe dataset is opgeslagen op mijn persoonlijke computer. R weet dan waar de externe dataset zich bevindt. Je kunt zien dat ik het bestand op

mijn Bureaublad (Desktop) heb opgeslagen.

`<-` zorgt ervoor dat R de externe dataset opslaat in een dataframe object die we `data_uitgaan` noemen. Dit noemen we de *assignment operator*.

Nu kun je de externe dataset zelf gaan laden. Zorg ervoor dat je de data hebt gedownload en opgeslagen op je Bureaublad. Pas vervolgens de code zo aan dat R de juiste locatie vindt van het bestand. Dit doe je door het juiste pad tussen aanhalingstekens aan de te passen (`"Hier_jouw_pad/data_uitgaan.rds"`). Je kunt op je computer naar de locatie van het bestand gaan (Bureaublad) en dan via rechtermuisknop en 'Als pad kopiëren' de exacte locatie van het bestand vinden en hier plakken. Let op dat je in R `/` gebruikt voor bestandslocaties in plaats van \. Dit moet je nog verder aanpassen. Zorg ervoor dat jouw code het voorbeeld volgt van het codeblok hierboven. Run vervolgens de code.

1. (Als je er echt niet uitkomt, kun je ook op het bestand van de externe dataset dubbelklikken en dit direct laden in R. Je krijg dan een pop-up met de vraag of je dit bestand direct wil laden in R. Klik op OK)
2. (Je kunt ook altijd Googlen of DuckDuckGo om uit te vinden hoe iets moet)

```
# laden data
data_uitgaan <- read_rds("Hier_jouw_pad/data_uitgaan.rds")
```

```
## Warning in readRDS(con, refhook = refhook): cannot open file
## 'Hier_jouw_pad/data_uitgaan.rds': No such file or directory
```

```
## Error in readRDS(con, refhook = refhook): cannot open the connection
```

Als het is gelukt, zul je rechtsboven in Rstudio een regel zien met `data_uitgaan` onder het tabblad Environment. Daar zie je ook hoeveel observaties en variabelen de dataset heeft.

Data inspecteren

De volgende stap na het laden van de data, is het inspecteren ervan. Hierdoor krijgen we een globaal idee van hoe onze dataset in elkaar zit. Hiervoor gebruiken we de functie `View()`. R opent dan het dataframe en je ziet de observaties en variabelen. Run onderstaande code. Scroll een beetje van onder naar boven en links en rechts en denk na over wat de waardes die je ziet kunnen betekenen.

```
# inspecteren van de data
View(data_uitgaan)
```

Variabelen beschrijven

Een eerste stap in analyse is dat we een beschrijving geven van de variabelen waarmee we werken. In dit practicum werken we met de volgende variabelen (en antwoordcategorieen):

`opleidingsniveau`: Het hoogst behaalde opleidingsniveau van de respondent

- laag / midden / hoog

`cannabis_ooit`: Of de respondent ooit cannabis (wiet of hasj) heeft gebruikt.

- ja / nee

`cannabis_frequentie`: frequentie waarmee respondent cannabis (wiet of hasj) heeft gebruikt het afgelopen jaar.

- één keer / minder dan maandelijks / eens per maand / een paar keer per maand / eens per week / een paar keer per week / (bijna) elke dag

`gevoelstemperatuur_cannabis`: welk gevoel een respondent bij cannabis in het algemeen heeft (de beoordeling ervan).

- score van 0 tot en met 100, waarbij 0 staat voor negatief en 100 voor positief.

Bedenk eerst wat het meetniveau is van deze variabelen. Geef je antwoord met korte uitleg hieronder:

`opleidingsniveau`: [Jouw antwoord hier]

`cannabis_ooit`: [Jouw antwoord hier]

`cannabis_frequentie`: [Jouw antwoord hier]

`gevoelstemperatuur_cannabis`: [Jouw antwoord hier]

Eenvoudige tabel

Met aantallen We gaan nu voor opleidingsniveau kijken wat de verdeling is op deze variabele. Dit doen we eerst middels een tabel. Daarvoor gebruiken we de functie `table()`. Hieronder zie je een voorbeeld voor de variabele `geslacht`.

Je vraagt hier een tabel op van een variabele via `table()`. Om die variabele te selecteren, moet je eerst het dataframe aangeven: `data_uitgaan`. Binnen dat dataframe geef je aan om welke variabele het gaat. Dit doe je middels de `$` operator. Deze zegt het ware: binnen `data_uitgaan`, neem de variabele `geslacht`: `data_uitgaan$geslacht`. Run nu de code hieronder.

```
# tabel van geslacht met aantallen
table(data_uitgaan$geslacht)
```

```
##
##    man  vrouw anders
##    178    170      2
```

Als je dit codeblok runt, zul je net als bij Meetniveas en visualisaties zien dat in de dataset 178 mannen, 170 vrouwen en 2 anders-identificerenden zitten.

Je zou nu zelf in staat moeten zijn een tabel voor `opleidingsniveau` te maken. Doe dat in het codeblok hieronder. Typ de code zelf hier onder `# tabel van opleidingsniveau met aantallen`. Vergeet niet de code te runnen.

```
# tabel van opleidingsniveau met aantallen
```

Als het is gelukt de tabel te maken, rapporteer dan hoeveel respondenten er voor elke categorie in de dataset zitten.

[Jouw antwoord hier]

Met proporties Als je de proporties van elke categorie van een variabele wil weten, kun je expliciet aangeven dat je proporties wil berekenen. Dit doe je door `|> prop.table()` toe te voegen. `prop.table()` is een functie om de proporties binnen een tabel te berekenen.

`|>` is een operator die je nog vaker zult tegenkomen. Het neemt een stuk code aan de linkerkant van de operator en geeft het als eerste argument aan de rechterkant. Dit wil zeggen dat `table(data_uitgaan$geslacht) |> prop.table()` hetzelfde is als `prop.table(table(data_uitgaan$geslacht))`. Als je dit nu nog niet snapt, is dat helemaal niet erg. Je zult gedurende de cursus gaan zien waarom het meerwaarde heeft om R code zo te schrijven. Hieronder staat een voorbeeld met de variabele `geslacht`. Run deze code.

```
# tabel van geslacht met proporties
table(data_uitgaan$geslacht) |> prop.table()
```

```
##
##         man       vrouw      anders
## 0.508571429 0.485714286 0.005714286
```

De output geeft nu proporties weer. De dataset bestaat dus (afgerond) uit 50,9% mannen, 48,6% vrouwen en 0,6% anders-identificerenden.

Maak nu een tabel van opleidingsniveau met proporties. Op basis van bovenstaande instructies kun je uitvogelen hoe dat moet. Typ de juiste code onder `# tabel van opleidingsniveau met proportie`sen run vervolgens de code.

```
# tabel van opleidingsniveau met proporties
```

Rapporteer nu de proporties voor elke categorie van de variabele `opleidingsniveau`.

[Jouw antwoord hier]

Staafdiagram

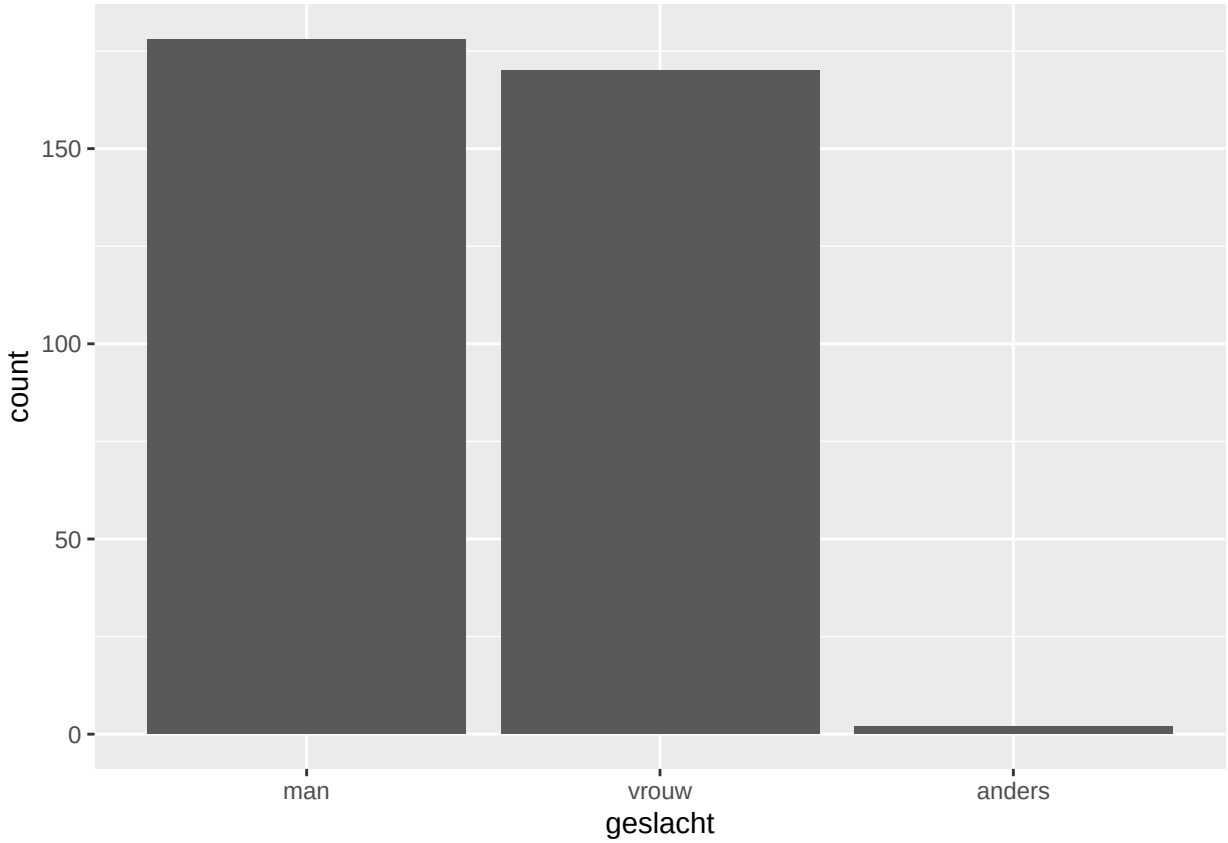
(Vanaf nu zeggen we niet meer 'run de code'. We gaan er vanuit dat je dit automatisch doet bij elk codeblok)

We gaan nu een staafdiagram maken van `opleidingsniveau`. De basis van datavisualisatie (het maken van figuren) is de functie `ggplot()`. Deze functie kunnen we, zoals je zult zien, uitbouwen met diverse componenten waardoor we figuren laag voor laag kunnen vormgeven.

De basis die we vaak gebruiken is `ggplot(data, aes(x = variabele))`. Hier moeten we bij `data` de naam van onze dataset invoeren en bij `variabele` de naam van onze variabele die we willen visualiseren. Vervolgens voegen we een geometrische laag (een type figuur) toe die aangeeft wat voor soort visualisatie het moet worden (bijvoorbeeld staafdiagram, puntenwolk, histogram etc.). We beginnen hier met een staafdiagram, dit gaat via `geom_bar()`. Voor `ggplot()` geldt dat we elke laag toevoegen middels een `+`-teken. Zo kunnen we een plot/visualisatie laag voor laag opbouwen.

Hieronder zie je weer het voorbeeld van de variabele `geslacht`. We hebben tevens de naam onze dataset ingevuld: `data_uitgaan`.

```
# staafdiagram van geslacht
ggplot(data_uitgaan, aes(x = geslacht)) +
  geom_bar()
```



Net als in het quantcrim handboek, zie je hier een basis staafdiagram van geslacht.

Maak nu een staafdiagram van `opleidingsniveau`. Typ de juiste code in het codeblok hieronder (en run deze code uiteraard).

```
# staafdiagram van opleidingsniveau
```

We gaan nu kijken hoe je een staafdiagram kan 'oppimpen'. ggplot biedt veel opties om figuren op een meer aantrekkelijke en duidelijke manier te presenteren. We gaan hier de achtergrond, de assenlabels en titel aanpassen.

`theme_bw()` staat voor 'thema in zwart-wit (black and white)'. Dit zorgt ervoor dat de achtergrond wit wordt en de staven zwart.

`ylab()` past het label van de y-as aan. Je moet het nieuwe label tussen aanhelingstekens zetten: `ylab("Aantal")`.

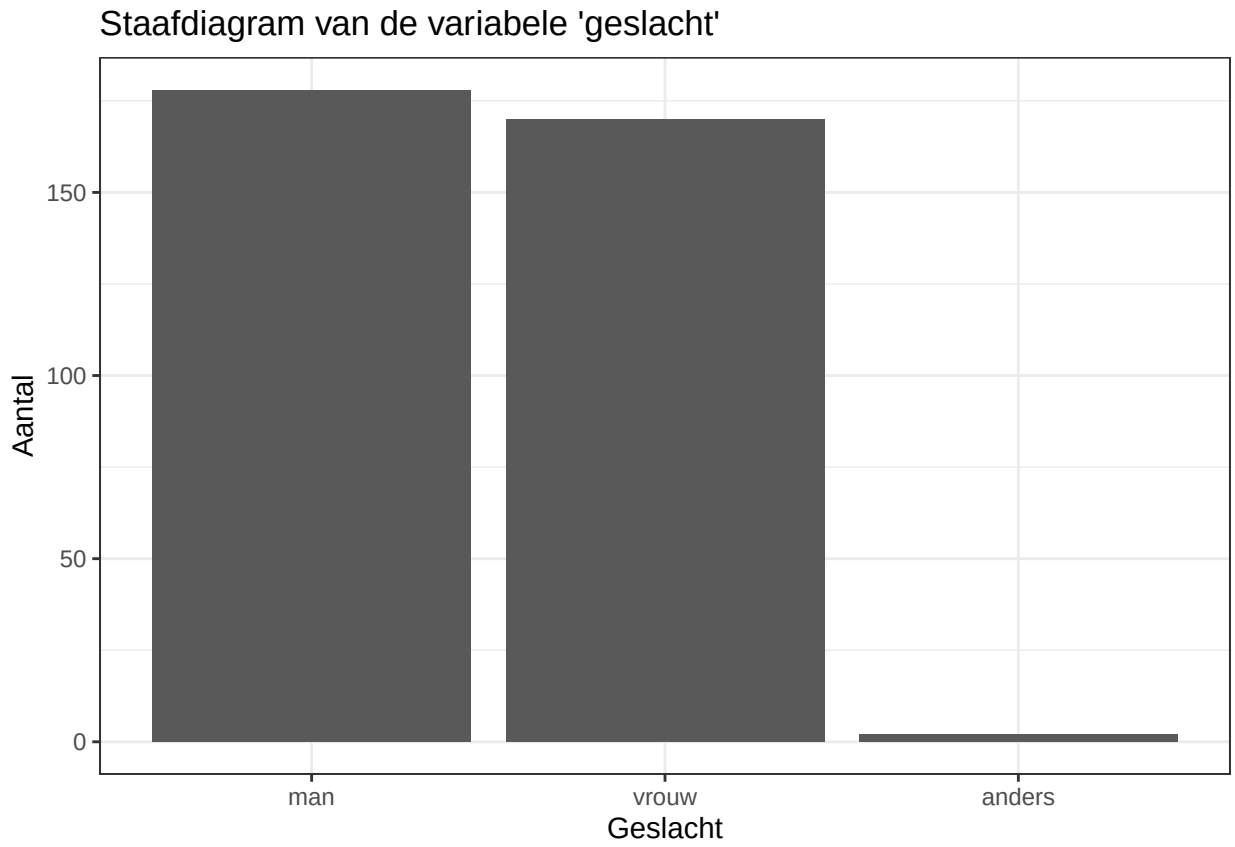
`xlab()` werkt hetzelfde als `ylab()`, maar dan voor de x-as: `xlab("Geslacht")`

`ggtitle()` geeft een titel aan de figuur, ook hier moet de tekst tussen aanhalingstekens: `ggtitle("Staafdiagram van de variabele 'geslacht'")` .

Met behulp van het `+`-teken voegen we alle lagen toe aan de figuur van geslacht.

```
# staafdiagram van geslacht met opmaak
ggplot(data_uitgaan, aes(x = geslacht)) +
  geom_bar() +
  theme_bw() +
  ylab("Aantal") +
  xlab("Geslacht") +
```

```
  ggtitle("Staafdiagram van de variabele 'geslacht'")
```



Dat ziet er al een stuk beter uit. Vergelijk deze opgemaakte staafdiagram met de basis staafdiagram van geslacht en kijk welke veranderingen er plaats hebben gevonden.

Pas nu de staafdiagram op dezelfde manier aan voor `opleidingsniveau`. Verander de achtergrond, labels van de assen en titel. Zorg dat deze passend zijn voor deze variabele. Breid de code uit die je eerder hebt gemaakt.

```
# staafdiagram van opleidingsniveau met opmaak
```

Variabelen hercoderen

Het komt in onderzoek vaak voor dat we een variabele willen hercoderen. Hercoderen betekent dat we waardes van een variabele nieuwe waardes geven, bijvoorbeeld om categorieën samen te voegen of data anders te presenteren.

Als voorbeeld kijken we naar frequentie van alcoholgebruik in het afgelopen jaar (`alcohol_frequentie`). De antwoorden zijn bij deze variabele hetzelfde als bij `cannabis_frequentie`. We vragen eerst een tabel op de aantallen per categorie.

```
# tabel van alcohol frequentie met aantallen
table(data_uitgaan$alcohol_frequentie)
```

```
##
##                een keer  minder dan maandelijks          eens per maand
##                       1                      12                      12
## een paar keer per maand           eens per week  een paar keer per week
```

```
##                      53                      86                     160
##        (bijna) elke dag
##                      16
```

Je kunt zien dat één respondent een keer alcohol heeft gedronken, 12 respondenten minder dan maandelijks, 12 respondenten eens per maand, etc.

We willen nu bijvoorbeeld weten welk aandeel van de respondenten wekelijks drinkt (en welk deel niet). Daarvoor kunnen we alle antwoorden op 'een keer' tot en met 'een paar keer per maand' samenvoegen, en alle antwoorden op 'eens per week' tot en met '(bijna) elke dag'. Dit doen we middels hercoderen.

We maken een nieuwe variabele aan, deze noemen we `alcohol_frequentie_her`.

We gebruiken `recode_values()` om te hercoderen. We vertellen R wat de originele variabele is (`alcohol_frequentie`) en dan per waarde bepalen we een nieuwe categorie. Dat doen we door steeds de originele categorie te nemen (bijvoorbeeld `"een keer"`), dan een ~-teken te plaatsen, en dan de nieuwe categorie aan te duiden (`"maandelijks of minder"`). Dit moeten we doen voor elke oorspronkelijke categorie. Dat is geïllustreerd in de code hieronder.

```
# hercoderen naar alcohol_frequentie_her
data_uitgaan$alcohol_frequentie_her <-
  recode_values(data_uitgaan$alcohol_frequentie,
                "een keer"                ~ "maandelijks of minder",
                "minder dan maandelijks"  ~ "maandelijks of minder",
                "eens per maand"          ~ "maandelijks of minder",
                "een paar keer per maand" ~ "maandelijks of minder",
                "eens per week"           ~ "wekelijks",
                "een paar keer per week"  ~ "wekelijks",
                "(bijna) elke dag"        ~ "wekelijks")
```

Vervolgens vragen we een tabel op met aantallen van de nieuwe variabele `alcohol_frequentie_her` om te checken of het hercoderen goed is gegaan.

```
# tabel van alcohol frequentie (gehercodeerd) met aantallen
table(data_uitgaan$alcohol_frequentie_her)
```

```
##
## maandelijks of minder             wekelijks
##                    78                   262
```

Je ziet bijvoorbeeld dat 'wekelijks' 262 respondenten heeft. Als je de originele categorieen optelt, kom je op $86 + 160 + 16 = 262$ respondenten. Dit toont dus dat het hercoderen is gelukt! (dit kun je ook checken voor de nieuwe categorie 'maandelijks of minder').

Ten slotte maken we een tabel met proporties om te zien welk deel wekelijks drinkt.

```
# tabel van alcohol frequentie (gehercodeerd) met proporties
table(data_uitgaan$alcohol_frequentie_her) |> prop.table()
```

```
##
## maandelijks of minder             wekelijks
##             0.2294118             0.7705882
```

77,1 procent van de respondenten drinkt dus wekelijks (van degenen die alcohol hebben gedronken het afgelopen jaar).

Je hebt nu genoeg gezien om hetzelfde voor `cannabis_frequentie` te doen. Doorloop dus dezelfde stappen:

1. Maak een tabel van `cannabis_frequentie`.

2. Hercodeer de variabele op dezelfde wijze als bij `alcohol_frequentie`. Om het wat makkelijker te maken, hebben we een deel van de code al gegeven. Je moet `originele_variabele` en `nieuwe_variabele` vervangen met de juiste code. Verder moet je de middelste drie categorieën nog hercoderen (vervang de `...`) .
3. Check of het hercoderen van de variabele goed is gegaan middels tabel voor de nieuwe variabele.
4. Maak een tabel met proporties voor de nieuwe variabele en rapporteer het aandeel dat wekelijks cannabis gebruikt.

Typ de code in de juiste codeblokken hieronder.

```
# tabel van cannabis frequentie met aantallen
```

```
# hercoderen naar cannabis_frequentie_her
nieuwe_variabele <-
  recode_values(originele_variabele,
                "een keer"                ~ "maandelijks of minder",
                "minder dan maandelijks"  ~ "maandelijks of minder",
                ... ,
                ... ,
                ... ,
                "een paar keer per week"  ~ "wekelijks",
                "(bijna) elke dag"        ~ "wekelijks")
```

```
## Error: '...' used in an incorrect context
```

```
# tabel van cannabis frequentie (gehercodeerd) met aantallen
```

```
# tabel van cannabis frequentie (gehercodeerd) met proporties
```

[Jouw antwoord hier]

Relaties tussen variabelen

In het laatste deel van de opdracht gaan we relaties tussen variabelen beschrijven. Daarbij kijken we tevens hoe we met missende waarden kunnen omgaan. Dit is wanneer respondenten een ontbrekende waarde hebben op een variabele.

Kwali vs. kwanti

We gaan eerst het verband tussen `cannabis_ooit` en `gevoelstemperatuur_cannabis` onderzoeken. Het is handig eerst de verdeling van deze variabelen afzonderlijk te bekijken.

Centrum- en spreidingsmaten Als voorbeeld nemen we twee vergelijkbare variabelen: `tabak_ooit` en `gevoelstemperatuur_tabak`. Deze variabelen meten respectievelijk of respondenten ooit hebben gerookt en wat hun gevoel is t.o.v. tabak (of ze deze drug positief of negatief waarderen).

Voor `tabak_ooit` vragen we een tabel op met aantallen. We voegen nu alleen het argument `useNA = "always"` toe om ook het aantal missende waarden te krijgen.

```
# tabel van ooit tabak gedronken (inclusief missende waarden)
table(data_uitgaan$tabak_ooit, useNA = "always")
```

```
##
##  nee   ja <NA>
##   74  276    0
```

De nul onder <NA> toont dat geen enkele respondent een missende waarde heeft (mooi nieuws!). 74 respondenten geven aan nooit gerookt te hebben en 276 wel.

Voor `gevoelstemperatuur_tabak` gebruiken we de functie `summary()`. Deze geeft ons een aantal centrum- en spreidingsmaten, en ook nog het aantal missende waarden.

```
# statistieken van gevoelstemperatuur tabak
summary(data_uitgaan$gevoelstemperatuur_tabak)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##    0.00   22.00   35.00   35.22   48.00  100.00      15
```

Je ziet dat de gemiddelde gevoelstemperatuur ten aanzien van tabak 35,2 is en de mediaan 35,0. De minimale score is 0 de maximale 100. Het aantal missings bedraagt 15. Dit is relatief laag ten opzichte van van de gehele dataset ($15/350 = 0,04$): 4%.

Maak nu een tabel voor `cannabis_ooit` waarin je ook de missings opvraagt en vraag tevens de centrum- en spreidingsmaten op voor `gevoelstemperatuur_cannabis`. Rapporteer daarna: het aantal personen dat ooit cannabis heeft gebruikt, de gemiddelde gevoelstemperatuur t.a.v. cannabis en het aantal missings per variabele. Typ de code in de volgende blokken.

```
# tabel van ooit cannabis gebruikt (inclusief missende waarden)
```

```
# statistieken van gevoelstemperatuur cannabis
```

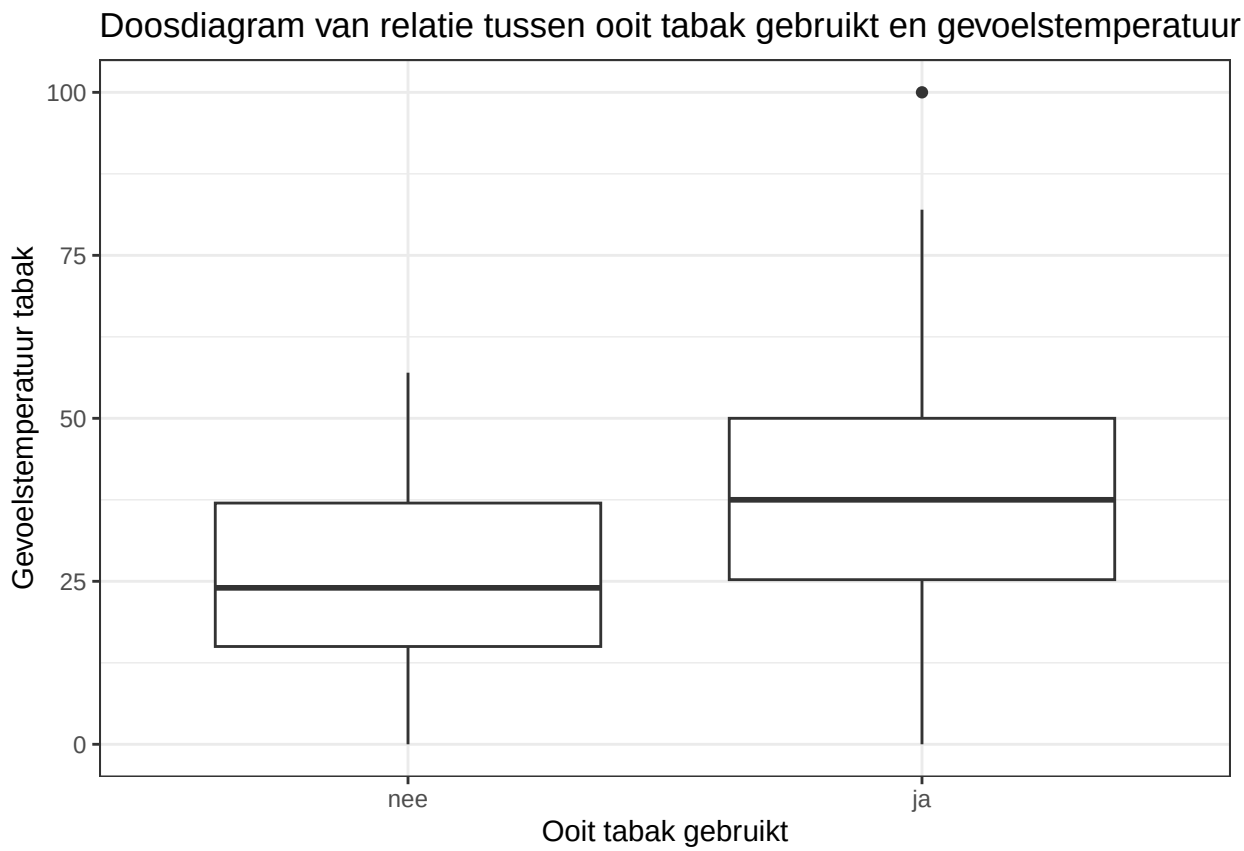
[Jouw antwoord hier]

Doosdiagram In de volgende stap maken we een doosdiagram waarin we de spreiding voor `gevoelstemperatuur_cannabis` apart bekijken voor mensen die wel en niet cannabis geprobeerd hebben (`cannabis_ooit`).

Dit doen we eerst voor tabak om een voorbeeld te geven. We kunnen weer `ggplot()` gebruiken om de doosdiagram te maken. Omdat we nu twee variabelen hebben (`tabak_ooit` en `gevoelstemperatuur_tabak`), moeten we naast `x = ...` ook `y = ...` invullen. We kiezen er in principe altijd voor om de kwalitatieve variabele op de x-as te zetten en de kwantitatieve op de y-as. Verder hebben we voor een doosdiagram `geom_boxplot()` nodig als geometrisch object. Ten slotte voegen we alvast alle opmaakopties toe die we eerder hebben gezien bij Staafdiagram: achtergrond, assenlabels en titel.

```
# doosdiagram voor relatie tussen ooit tabak gebruikt en gevoelstemperatuur
ggplot(data_uitgaan, aes(x = tabak_ooit, y = gevoelstemperatuur_tabak)) +
  geom_boxplot() +
  theme_bw() +
  xlab("Ooit tabak gebruikt") +
  ylab("Gevoelstemperatuur tabak") +
  ggtitle("Doosdiagram van relatie tussen ooit tabak gebruikt en gevoelstemperatuur")
```

```
## Warning: Removed 15 rows containing non-finite outside the scale range
## (`stat_boxplot()`).
```



Je ziet dat de mediaan van gevoelstemperatuur voor respondenten die ooit hebben gerookt een stuk hoger ligt dan bij respondenten die dit nooit hebben gedaan. Eerstgenoemde groep staat dus positiever tegenover tabak dan laatstgenoemde groep; visueel gezien lijkt het ongeveer 13 punten te schelen. Ook is de spreiding in gevoelstemperatuur wat groter bij de groep die ooit tabak heeft gebruikt.

(Je ziet ook dat R een waarschuwing geeft over hoeveel rijen (d.w.z., observaties) er zijn verwijderd, dit komt overeen met het aantal wat we eerder hebben vastgesteld)

Maak nu eenzelfde doosdiagram voor de relatie tussen `cannabis_ooit` en `gevoelstemperatuur_cannabis`. Zorg dat alle elementen (achtergrond, assenlabels en titel) op orde zijn. Beschrijf vervolgens de relatie tussen deze variabelen m.b.t. het centrum en de spreiding. Type de code hieronder.

```
# doosdiagram voor relatie tussen ooit cannabis gebruikt en gevoelstemperatuur
```

[Jouw antwoord hier]

Kwali vs. kwali

Voor dit onderdeel onderzoeken we de relatie tussen `opleidingsniveau` en `cannabis_frequentie_her`. Hiermee krijgen we inzicht in welke mate cannabisconsumptie verschilt naar opleidingsniveau. We hebben eerder al bekeken wat de verdeling is op deze variabelen afzonderlijk (onder Variabelen beschrijven en Variabelen hercoderen).

Kruistabel Ter illustratie kijken we eerst naar de relatie tussen `opleidingsniveau` en `alcohol_frequentie_her`. We kunnen een kruistabel maken met `table()` waarin we twee variabelen toevoegen (in plaats van één zoals tot nu toe). We zetten `opleidingsniveau` in de kolommen en `alcohol_frequentie_her` in de rijen. Zie het codeblok hieronder.

```r
# kruistabel tussen alcohol frequentie (gehercodeerd) en opleidingsniveau
table(data_uitgaan$alcohol_frequentie_her, data_uitgaan$opleidingsniveau,
      useNA = "always")
```

```
##
##                         laag midden hoog <NA>
##   maandelijks of minder    2     23   52    1
##   wekelijks                7     74  177    4
##   <NA>                     1      7    1    1
```

Je ziet nu de verdeling van respondenten in aantallen over de combinatie van categorieën. Zo zijn er bijvoorbeeld 74 respondenten met een middelbaar opleidingsniveau die wekelijks drinken.

Het is vaak inzichtelijker om naar de proporties (of percentages) per kolom te kijken. Hiervoor voegen we weer `prop.table()` toe. Die berekent dan de proportie per categorie over het totaal aantal cellen. We willen echter alleen de kolomproporties weten. Daarvoor moeten we een `2` toevoegen aan `prop.table()`. (met een `1` krijgen we de rijproporties).

```r
# kruistabel tussen alcohol frequentie (gehercodeerd) en opleidingsniveau met kolomproporties
table(data_uitgaan$alcohol_frequentie_her, data_uitgaan$opleidingsniveau,
      useNA = "always") |> prop.table(2)
```

```
##
##                              laag      midden        hoog        <NA>
##   maandelijks of minder 0.200000000 0.221153846 0.226086957 0.166666667
##   wekelijks             0.700000000 0.711538462 0.769565217 0.666666667
##   <NA>                  0.100000000 0.067307692 0.004347826 0.166666667
```

De kruistabel met kolomproporties toont dat 71,2% van de middelbaar opgeleiden wekelijks drinkt, tegenover 22,1% die maandelijks of minder drinken.

Maak nu soortgelijke tabellen voor de relatie tussen `opleidingsniveau` en `cannabis_frequentie_her`. Zet `opleidingsniveau` in de kolommen en `cannabis_frequentie_her` in de rijen. Typ de relevante code in de blokken hieronder.

(als het bij Variabelen hercoderen niet is gelukt om `cannabis_frequentie_her` te maken, kun je de variabele `cannabis_frequentie_her2` gebruiken. Deze zit al in de dataset).

```r
# kruistabel tussen cannabis frequentie (gehercodeerd) en opleidingsniveau
```

```r
# kruistabel tussen alcohol frequentie (gehercodeerd) en opleidingsniveau met kolomproporties
```

Gestapelde staafdiagram

Data prepareren We gaan nu een visualisatie maken van de relatie tussen `opleidingsniveau` en `cannabis_frequentie_her`.

Omdat `ggplot()` niet automatisch de kolomproporties berekend, moeten we dat eerst zelf doen. Als voorbeeld nemen we wederom de relatie tussen `opleidingsniveau` en `alcohol_frequentie_her`.

Voor de visualisatie maken we eerst een apart dataframe getiteld `opleiding_alcohol`. Hierin slaan we de kolomproporties op die we zelf berekenen (vergelijkbaar met wat we net hebben gedaan bij Kruistabel.

We gebruiken de functie `summarise()`. Zoals de naam al zegt, kun je deze functie gebruiken om data samen te vatten om bijvoorbeeld een gemiddelde of percentage te berekenen.

Met de functie `n()` zeggen we tegen R: tel het aantal respondenten per categorie van de aangegeven variabelen (`opleidingsniveau` en `alcohol_frequentie_her`).

Daarvoor moeten we de data eerst 'groeperen' op deze variabelen met de functie `group_by()`, dan weet R voor welke voor welke combinatie van variabelen we de proporties willen berekenen. Het aantal respondenten per categorie-combinatie slaan we op in de variabele `aantal`.

Dit staat in onderstaande code. We voegen meteen `View()` toe zodat het resulterende dataframe `opleiding_alcohol` kunt bekijken. We gebruiken de `|>`-operator om stap voor stap aan te geven wat we willen doen.

```
# dataframe maken van relatie opleidingsniveau-alcohol frequentie
opleiding_alcohol <- data_uitgaan |>
  group_by(opleidingsniveau, alcohol_frequentie_her) |>
  summarise(aantal = n())
```

```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by opleidingsniveau and
##   alcohol_frequentie_her.
## i Output is grouped by opleidingsniveau.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(opleidingsniveau, alcohol_frequentie_her))` for
##   per-operation grouping (`?dplyr::dplyr_by`) instead.
```

```
View(opleiding_alcohol)
```

Er zijn nu nog twee dingen die we moeten doen.

Ten eerste zie je dat er ook rijen zijn waarin de aantallen voor `NAs` zijn berekend. Hier zijn we niet zo in geïnteresseerd. Deze halen we weg middels de functie `filter()`.

`!is.na(opleidingsniveau) & !is.na(alcohol_frequentie_her)` zegt: behoud alle rijen met `opleidingsniveau` en `alcohol_frequentie_her` die geen `NAs` bevatten. `!is.na()` betekent: is niet missend.

We verwijderen de missende waarden *voorafgaand* aan berekenen van de de aantallen. Vandaar dat we toevoegen onder de regel `group_by(opleidingsniveau, alcohol_frequentie_her)`.

```
# dataframe maken van relatie opleidingsniveau-alcohol frequentie
opleiding_alcohol <- data_uitgaan |>
  group_by(opleidingsniveau, alcohol_frequentie_her) |>
  filter(!is.na(opleidingsniveau) & !is.na(alcohol_frequentie_her)) |> # hier invoegen filter()
  summarise(aantal = n())
```

```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by opleidingsniveau and
##   alcohol_frequentie_her.
## i Output is grouped by opleidingsniveau.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(opleidingsniveau, alcohol_frequentie_her))` for
##   per-operation grouping (`?dplyr::dplyr_by`) instead.
```

```
View(opleiding_alcohol)
```

Je ziet nu dat in `opleiding_alcohol` de rijen met `NAs` zijn verwijderd.

Als laatste maken we een nieuwe variabele `proportie` aan. Hiervoor moeten we de proportie berekenen. Dit kan vrij eenvoudig via `aantal/sum(aantal)`. Omdat de data al gegroepeerd, weet R dat de som (functie `sum()`) moet worden genomen over alle rijen van de eerste variabele `opleidingsniveau`. Met de functie `mutate()` kan een nieuwe variabele worden aangemaakt binnen deze manier van werken

```
# dataframe maken van relatie opleidingsniveau-alcohol frequentie
opleiding_alcohol <- data_uitgaan |>
```

```
  group_by(opleidingsniveau, alcohol_frequentie_her) |>
  filter(!is.na(opleidingsniveau) & !is.na(alcohol_frequentie_her)) |> # hier invoegen filter()
  summarise(aantal = n()) |>
  mutate(proportie = aantal/sum(aantal))
```

```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by opleidingsniveau and
##   alcohol_frequentie_her.
## i Output is grouped by opleidingsniveau.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(opleidingsniveau, alcohol_frequentie_her))` for
##   per-operation grouping (`?dplyr::dplyr_by`) instead.
```

```
View(opleiding_alcohol)
```

Je ziet dat nu een variabele (a.k.a) kolom die `proportie` is toegevoegd aan de dataset met de proporties.

Herhaal deze stappen nu zelf, maar dan voor de variabelen `opleidingsniveau` en `cannabis_frequentie_her`. Noem het dataframe `opleiding_cannabis`. Check of jouw dataframe vergelijkbaar is met het voorbeeld hierboven middels `View()`. Typ de code hieronder.

(Nogmaals, je kunt `cannabis_frequentie_her2` gebruiken als het hercoderen van `cannabis_frequentie_her` eerder niet is gelukt)

```
# dataframe maken van relatie opleidingsniveau-cannabis frequentie
```

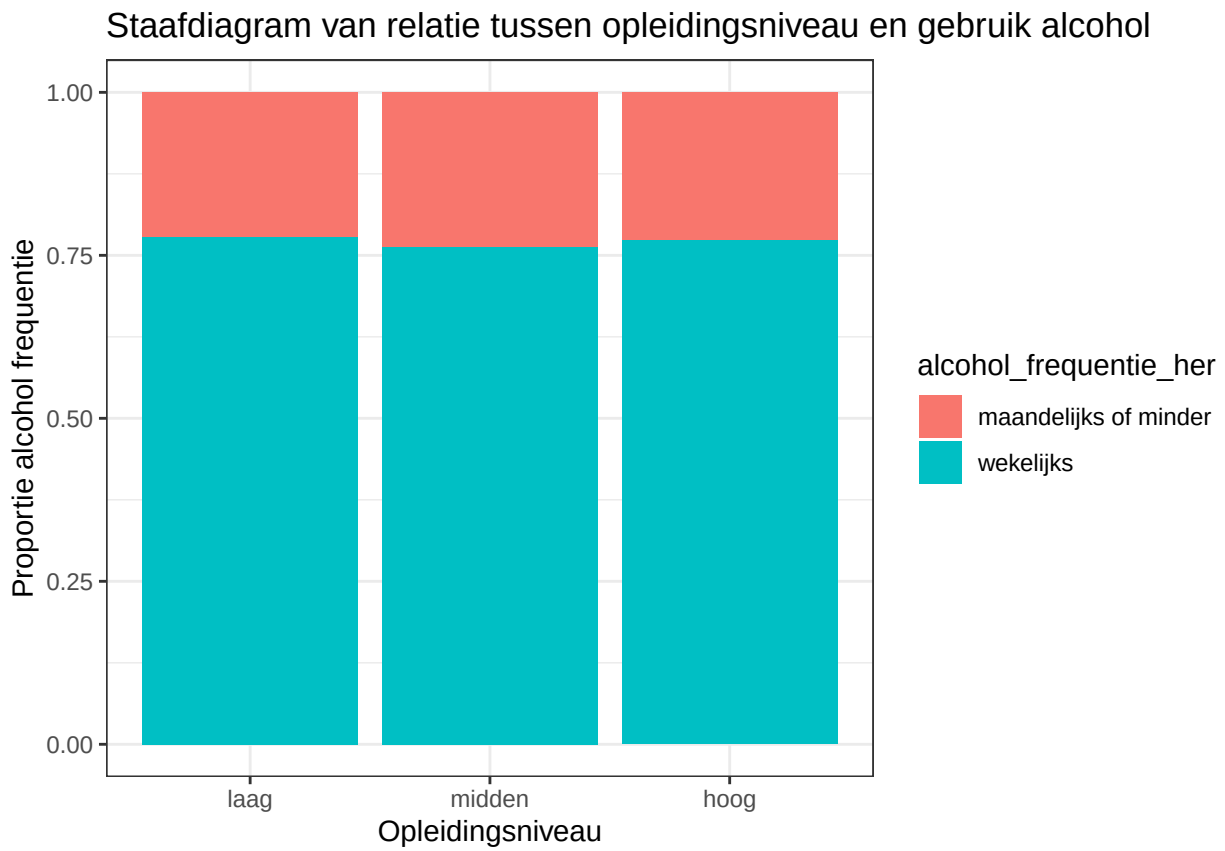
Staafdiagram maken We gaan nu de data van `opleiding_alcohol` visualiseren. Daarvoor gebruiken we de inmiddels bekende functies binnen `ggplot2` (de naam van het package). Omdat we hier met een iets meer complexe figuur te maken hebben, moeten we wel nog een aantal dingen toevoegen.

Ten eerste willen op de x-as `opleidingsniveau` plotten tegen de `proportie` van mate van alcoholgebruik. We moeten die proporties wel linken aan welke categorie van `alcohol_frequentie_her` het betreft (wekelijks of minder dan maandelijks). Dit doen we door naast `x = opleidingsniveau` en `y=proportie` ook `fill = alcohol_frequentie_her` toe te voegen `aes()`. Daardoor weet R als het ware hoe de staven moeten worden 'gevuld'.

Omdat we verder met een andere datastructuur werken (samengevatte data `opleiding_alcohol` i.p.v. oorspronkelijke data `data_uitgaan`), moeten we hier `geom_col()` gebruiken i.p.v. `geom_bar()` zoals bij simpele Staafdiagram.

Dit staan in de code hieronder.

```
# staafdiagram maken voor relatie opleidingsniveau-alcohol frequentie
ggplot(opleiding_alcohol,
       aes(x = opleidingsniveau, y = proportie, fill = alcohol_frequentie_her)) +
  geom_col() + #
  theme_bw() +
  ylab("Proportie alcohol frequentie") +
  xlab("Opleidingsniveau") +
  ggtitle("Staafdiagram van relatie tussen opleidingsniveau en gebruik alcohol")
```

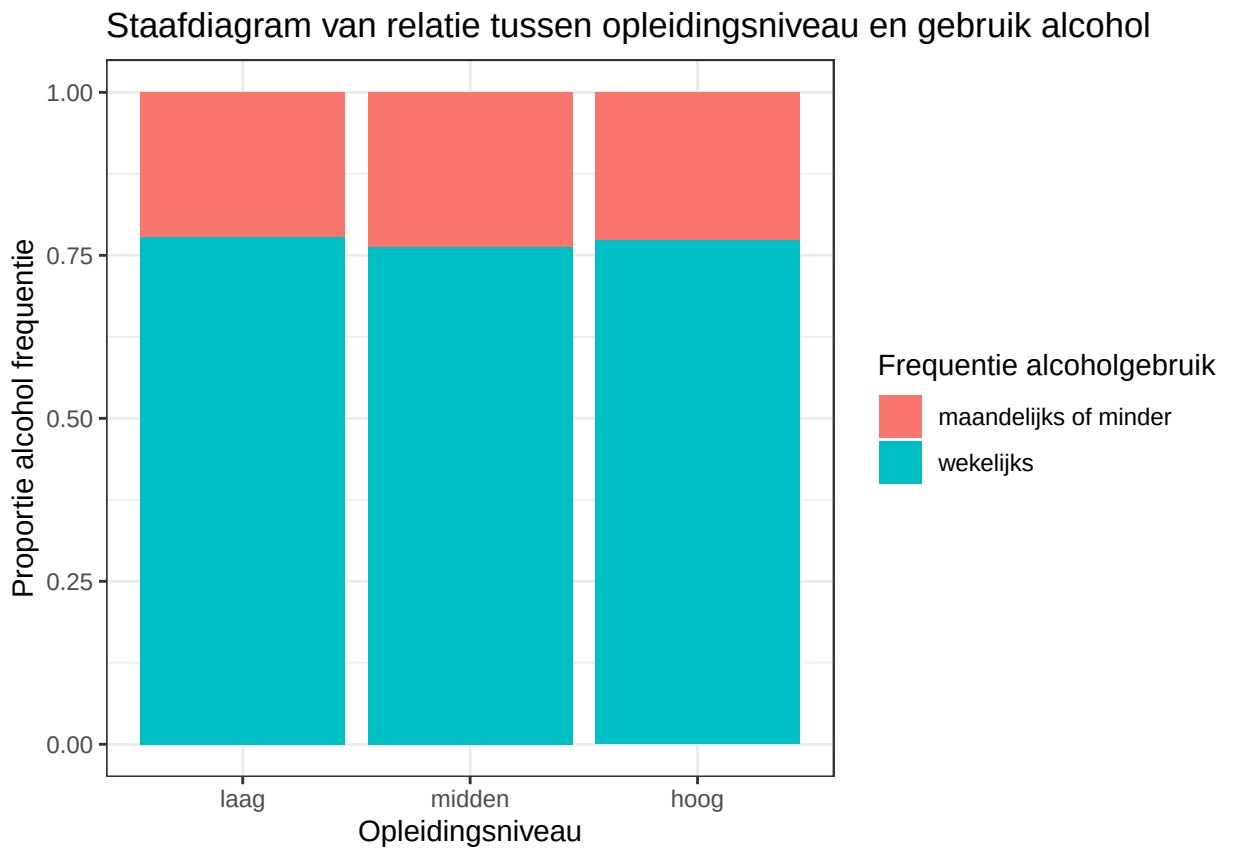


(haal de code `, fill = alcohol_frequentie_her` eens weg en run het codeblok opnieuw. Bedenk voor jezelf waarom je nu dit verschil ziet)

De staafdiagram toont duidelijk wat we eerder al zagen bij de kruistabel. Ongeveer driekwart van de respondenten drinkt wekelijks alcohol en er zijn verder nauwelijks verschillen tussen opleidingsniveaus hierin.

Als laatste *finetuning* kunnen we nog de titel van de legenda aanpassen, deze draagt nu de naam van de variabele `alcohol_frequentie_her`. Dit veranderen we door weer een laag toe te voegen middels de functie `labs()`. Omdat we hier `fill` gebruiken om de categorieen binnen de staven te identificeren, moeten we de code schrijven als `labs(fill = "Frequentie alcoholgebruik")`.

```
# staafdiagram maken voor relatie opleidingsniveau-alcohol frequentie
ggplot(opleiding_alcohol,
       aes(x = opleidingsniveau, y = proportie, fill = alcohol_frequentie_her)) +
  geom_col() + #
  theme_bw() +
  ylab("Proportie alcohol frequentie") +
  xlab("Opleidingsniveau") +
  ggtitle("Staafdiagram van relatie tussen opleidingsniveau en gebruik alcohol") +
  labs(fill = "Frequentie alcoholgebruik")
```



De visualisatie is nu helemaal compleet! Op basis van dit voorbeeld kun je hetzelfde doen voor de relatie tussen `opleidingsniveau` en `cannabis_frequentie_her`. Maak nu een soortgelijke figuur als hierboven met alle genoemde elementen. Geef vervolgens een interpretatie van de relatie.

```
# staafdiagram maken voor relatie opleidingsniveau-cannabis frequentie
```

[Jouw antwoord hier]

Opdracht inleveren

Je eerste practicumopdracht zit erop! Je hebt al een hoop geleerd over hoe je data laadt, bewerkt en visualiseert in R, en over verdelingen en relaties van diverse variabelen.

Als laatste moet je de opdracht inleveren via Canvas. De deadline hiervoor is **vrijdag 23:59**. Om de opdracht in te leveren, moet je er eerst een pdf-bestand van maken. Zoek hierboven naar 'Knit' (naast het bol garen iccontje), klik hierop het omgedraaide driehoekje en dan op 'Knit to PDF'. R maakt dan een pdf van je opdracht en slaat deze op in dezelfde map op je computer waar het .Rmd bestand staat. Lever dit pdf-bestand vervolgens in.

Ajeto!